

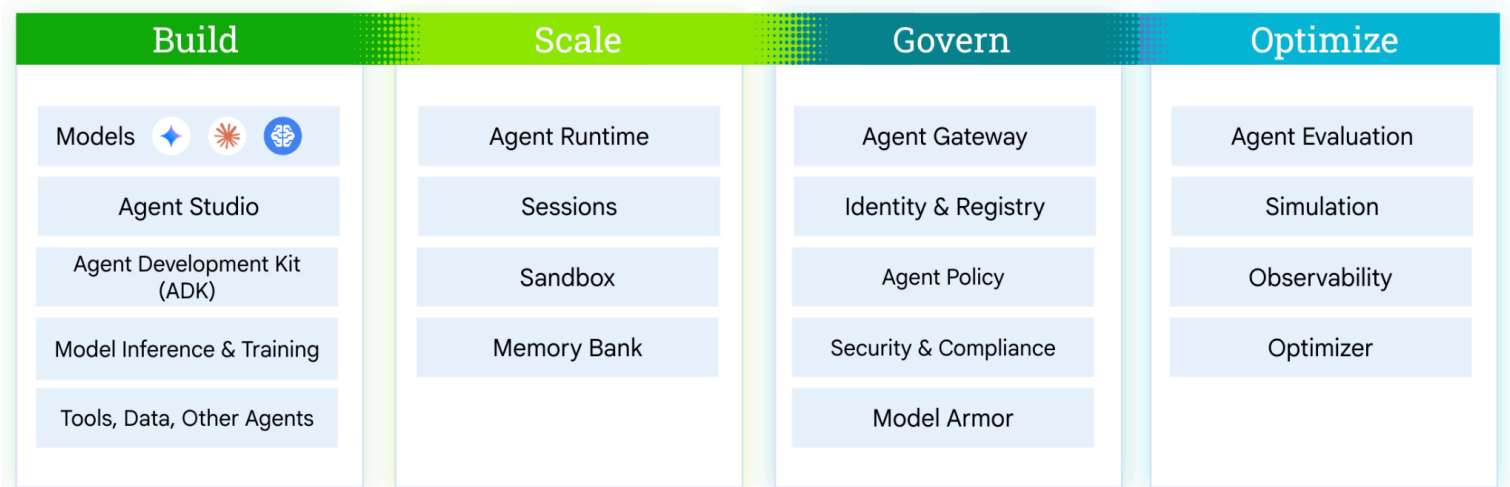


Onboarding Guide to Gemini Enterprise Agent Platform

Eric Dong Apr 22, 2026

Overview

Gemini Enterprise Agent Platform, the next evolution of Google Cloud **Vertex AI**, provides developers with a comprehensive environment to build, scale, govern, and optimize autonomous AI agents.



This quick-start guide accelerates your onboarding to Agent Platform, taking you step-by-step from initial environment setup to the successful deployment of autonomous AI agents.

Table of Contents

[Onboarding to Agent Platform](#)

- [Quick Start 1: Create Applications from Prompts](#)
- [Quick Start 2: Create and Deploy an Agent](#)
- [Quick Start 3: Build an agent with ADK and Agents CLI](#)

[API Enablement & Configuration](#)

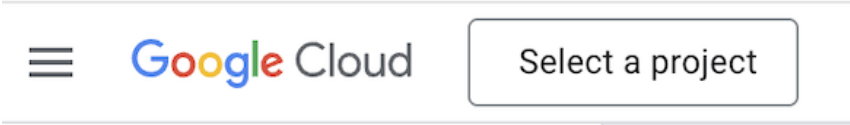
1. [Initial Project Setup](#)
2. [Install and initialize the Google Cloud CLI](#)
3. [Enable APIs](#)
4. [Authenticate your environment](#)
5. [SDK Initialization and Configuration](#)
 - [Google Gen AI SDK](#)
 - [Vertex AI SDK](#)
 - [ADK](#)

[Additional Resources](#)

Onboarding to Agent Platform

For rapid testing and experimentation, follow these steps to access Agent Platform via Google Cloud console:

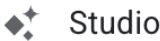
1. **Access Google Cloud console:** Navigate to <https://console.cloud.google.com/agent-platform/overview>.
2. **Select your project:** Select an existing project or create a new project.



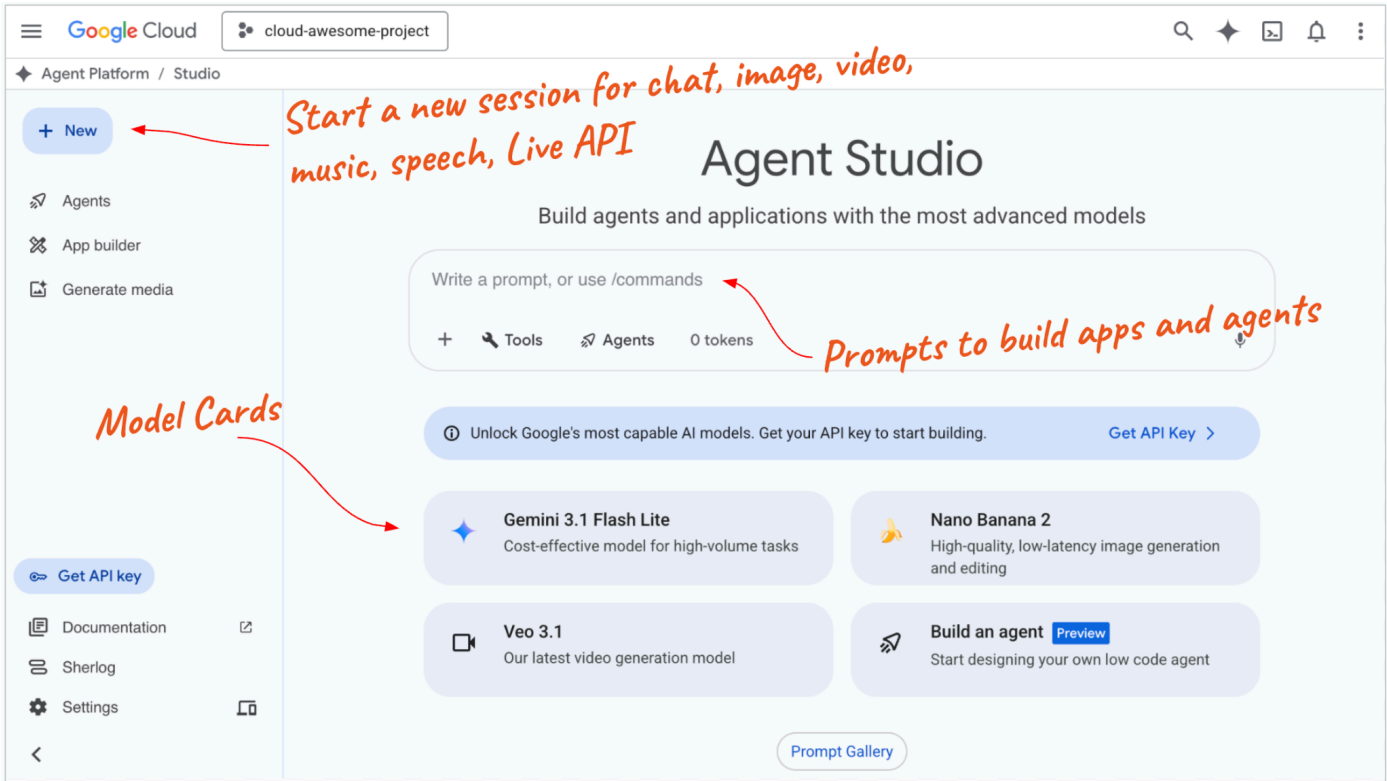
3. **Enable APIs:** Select the **Enable APIs** to enable required APIs.



4. **Access Agent Studio:** Select Studio from the left side options menu.



5. You are now in Agent Studio. You can use **Agent Studio** to design, test, and manage prompts for Gemini models and third-party models. You can start a session for **Chat, Image, Video, Music, Speech and Live API**.



Quick Start 1: Create Applications from Prompts

In this task, you'll see how quickly you can take an idea for an AI assistant and turn it into a working prototype using **Agent Studio**. You'll focus on our insurance use case: creating a prompt that helps an insurance professional summarize client information for a risk analysis report and then prepare this prompt as a simple application.

1. In Agent Studio, on the top left, select **+ New > Chat**. The prompt editor page opens.
2. On the top right, click **Untitled Prompt** and rename your prompt to **Insurance Risk Summary - Prototype**.

3. On the Model settings, ensure that model **gemini-3.1-pro-preview** is selected.
4. In the **System instructions** box, enter the following to give your AI assistant a role relevant to our insurance scenario:

You are an expert AI assistant for an insurance underwriting department.
Your primary goal is to help underwriters by accurately and concisely summarizing client information and highlighting potential risk factors.
Maintain a professional and objective tone.
Focus only on the information provided in the prompt. Do not invent details.

5. In the main prompt area, paste the following:

Customer Notes for 'SafeHarbor Warehousing':
"The applicant is seeking coverage for their 50,000 sq ft warehouse. The business is 5 years old. The building is a concrete tilt-up structure, originally built in 2010. They store a variety of non-hazardous dry goods.
Fire safety measures include a full sprinkler system, a centrally monitored fire alarm, and documented annual inspections by a certified third party.
Security measures include a 24/7 centrally monitored burglar alarm, comprehensive security camera coverage of the interior and exterior, a fully fenced perimeter, and nightly patrols by a contracted security guard service.
The company reports no major property or liability losses in their 5-year history. They have specifically asked to ensure their new automated shelving and retrieval system, installed last month, is adequately covered under the policy."

Your Task:

1. Briefly summarize the key details of the 'SafeHarbor Warehousing' business and its existing safety measures.
2. Based **only** on the notes provided, identify any immediate questions an underwriter should ask or potential risk factors they should consider further. Present the summary first, then the questions/risk factors as bullet points.

6. Click the **Submit (Enter)** arrow button or press ENTER. Review the model's response.
7. On the top right, next to your prompt name, click the **Save** button.
8. In the **Save prompt** dialog, click **Save**.

Note: It may take a few minutes for your first prompt to save.

9. Once your prompt has been saved, you'll explore how this drafted prompt can be turned into a prototype application. At the top right of the page, click the **Code** button.
10. From the menu that appears, select **Deploy > Cloud Run > Deploy as app**.

11. If the **Enable Cloud Run and required APIs** dialog appears, click **Enable required APIs**.

12. In the **Create a web app from the current prompt** dialog that appears:

- a. Check the Acknowledgement for publicly deploying your app.
- b. Click **Create App**.

13. If the **Grant permissions to the service account** dialog appears, click **Grant all**.

Note: The deployment process will begin and might take a few minutes.

Note: The deployment process may occasionally fail on the first attempt. This typically happens if the underlying permissions for the build service have not fully propagated when the deployment starts. If you see a "Failed" status in the Manage web app dialog, please follow these steps to retry:

1. Wait for approximately one minute to allow all services and permissions to initialize.
2. In the "Manage web app" dialog, click the Update app button.
3. A confirmation box will appear. Click Confirm to start the update process.

14. Once completed, click the **Close** button in the **Manage web app** box.

15. To open your newly deployed application, click **Code**, and then select **Deploy > Cloud Run > Open app**.

16. You should now see a page titled **Welcome to Agent Platform Gen AI App!** with your prompt title, **Insurance Risk Summary - Prototype**, displayed.

17. In the **Chatbot** section, in the "Type a message..." input field at the bottom, enter a new test message. For example:

New Customer Inquiry:

"Applicant 'Coastal Goods Delivery' has a fleet of 10 delivery vans, all equipped with GPS and telematics. They operate within a 100-mile radius of their depot. Drivers undergo annual safety training. They had one minor fender bender last year, no injuries, \$1500 damages. What are the primary risk considerations?"

Please summarize key points and identify potential risks.

18. Click the **submit** arrow button to send your message to the application.

19. Observe the response from your deployed app. It should process your input based on the logic and system instructions you defined in Agent Studio.

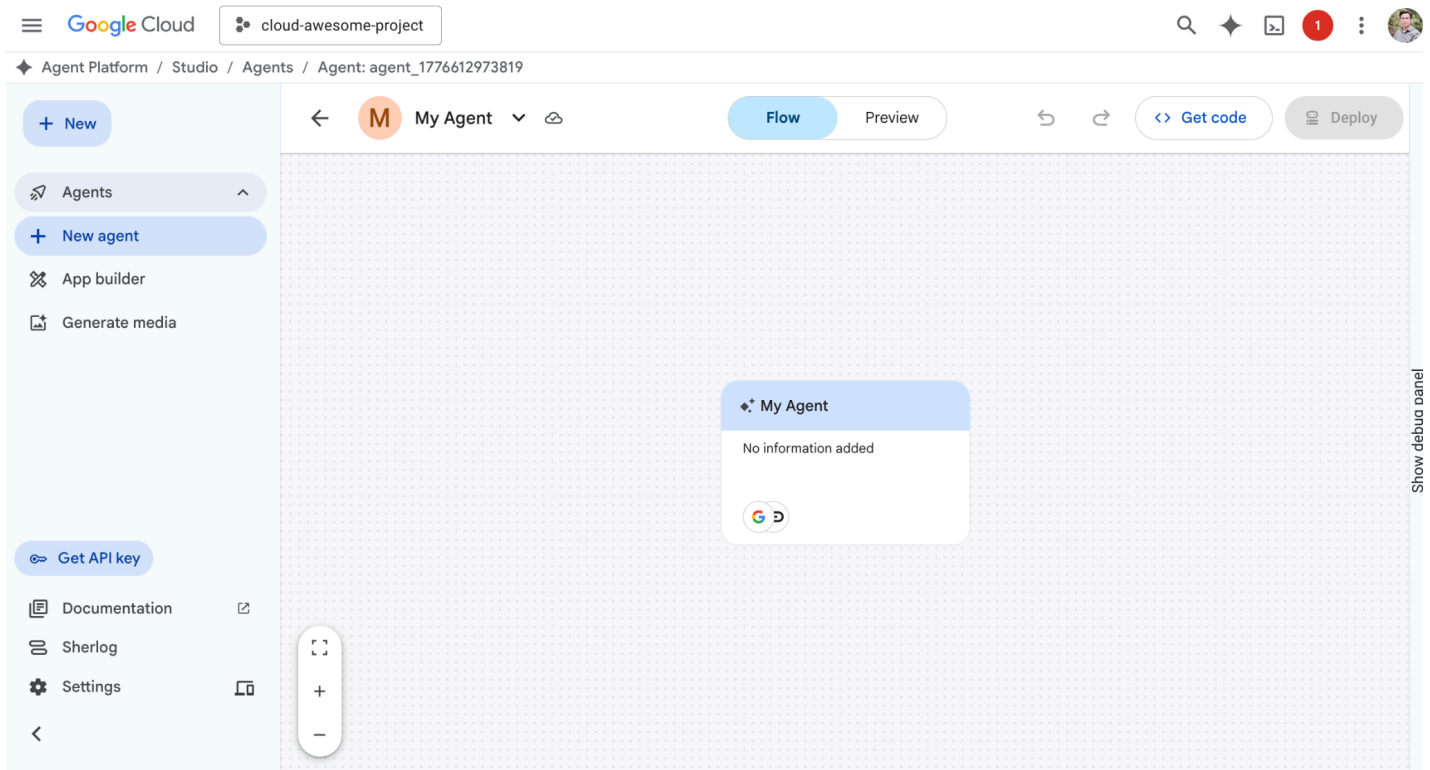
20. You have now completed the full cycle:

- a. Designed a prompt in Agent Studio.
- b. Deployed it as a serverless application using Cloud Run with a few clicks.
- c. Directly opened and interacted with your generative AI model through a web interface. This demonstrates the power of Agent Studio for rapid prototyping and deployment of generative AI capabilities.

Quick Start 2: Create and Deploy an Agent

In this task, you'll learn how you can create an agent using Agent Designer in Agent Studio, and deploy the agent to Agent Runtime. Agent Designer is a low-code visual designer that lets you design and test agents in the Google Cloud console. You can experiment with your agent in Agent Designer before transitioning development to code using [Agent Development Kit](#).

1. In Agent Studio, on the top left, select **Agent**. The Agent Designer page opens.
2. Click **Create agent** to open the Agent Designer canvas for a new agent.



3. Design your agent in the Agent Designer canvas. You can switch between the following tabs:



- **Flow:** Create the main agent and subagents by using a visual representation of your agent's workflow and control logic.
 - **Preview:** Test the agent's functionality and response as you build your agent. Chat with your agent to test its capabilities.
4. On the Flow tab, click **My agent** to open the **Details** panel for that agent.
 5. In the **Description** box, enter the following to give a description of your agent.

A helpful assistant for user questions.

6. In the **instruction** box, enter the following to provide instructions to your agent.

7. The following tools have been pre-configured by default in your agent.
 - **Google Search:** Enables the agent to perform web searches using Google Search.
 - **URL context:** Enable the model to analyze URLs from prompts sent to the agent.

Note: You can click **+** to add more tools.

8. Click **Get code** to see your agent code.

Note: You can copy and paste the code to a code editor of your choice. See [Agent Development Kit tutorials](#) to continue developing and deploying your agent.

9. Click **Deploy** to deploy the agent to Agent Runtime.
10. When the Deploy to an Agent Runtime instance dialog box appears, click **Deploy**.

Note: It may take a few minutes to deploy the agent.

Quick Start 3: Build an agent with ADK and Agents CLI

This guide demonstrates how to build, evaluate, and deploy a prototype AI agent using the [Agent Development Kit \(ADK\)](#) with the [Agents CLI](#) in Agent Platform. The ADK is an open-source, code-first framework for building sophisticated AI agents. The Agents CLI provides a unified, machine-readable interface for AI development tools to interact with the ADK, enabling end-to-end agent lifecycle management.

By using the Agents CLI with your preferred AI-powered development tool (such as Gemini CLI, Claude Code, or Codex), you can use natural language prompts to define, test locally, and deploy a prototype agent to a Google Cloud runtime. This tutorial guides you through creating a "caveman compressor" agent, which transforms verbose text into concise summaries, inspired by [caveman](#).

Before you begin

Before you begin, you need to ensure to have an AI-powered development tool installed, such as [Antigravity](#), [Gemini CLI](#), Claude Code, Codex, or any other AI development tool. You need this type of tool to interact with Agents CLI, which gives it the ability to search and retrieve information from Google's official developer documentation and enables it to perform agent development lifecycle tasks.

Setup

- Install [uv](#), a Python package installer. For instructions, see the [uv installation guide](#).

Scaffold Project

Then open your coding agent — [Gemini CLI](#), [Antigravity](#), [Claude Code](#), [Codex](#), or any other. Tell your coding agent:

```
None

"Use agents-cli to build a caveman-style agent that compresses verbose text into
terse, technical grunts"
```

Your coding agent activates the `google-agents-cli-workflow` and `google-agents-cli-scaffold` skills. It will:

- Ask clarifying questions (deployment target, safety constraints, etc.)
- Write a `DESIGN_SPEC.md` capturing the agent's purpose
- Scaffold the project:

```
None

agents-cli create caveman-agent --prototype --yes
cd caveman-agent && agents-cli install
```

You now have a working project with boilerplate agent code, tests, and eval sets.

Build Agent

Your coding agent edits `app/agent.py` — replacing the default agent with your caveman compressor. It uses the `google-agents-cli-adk-code` skill for ADK patterns.

The agent definition ends up looking something like:

```
Python
root_agent = Agent(
    name="caveman_agent",
    model=Gemini(model="gemini-flash-latest"),
    instruction="""You caveman compressor. Human give long words,
you make short. Rules:
- No articles. No filler. No fluff.
- Short grunts. Simple words.
- Keep technical terms but grunt around them.
- Funny but meaning stays.

Example input:  "I would like to deploy the application to production"
Example output: "Me deploy. Production. Now."
""",
)
```

Your coding agent then smoke-tests it:

```
None
agents-cli run "Please help me understand the deployment options available for my project"
```

Output:

> "Deploy options: Agent Runtime, Cloud Run, GKE. Pick one. Ship."

Evaluate Agent

Tell your coding agent:

```
None
"Write evals for the caveman agent and run them"
```

Your coding agent activates the `google-agents-cli-eval` skill and:

- Creates `tests/eval/evalsets/caveman.evalset.json` with test cases (compression quality, technical term preservation, caveman tone)
- Configures LLM-as-judge criteria in `tests/eval/eval_config.json`
- Runs the evaluation:

```
None
agents-cli eval run
```

If cases fail, tell your coding agent what to fix:

> "The response to the greeting test is too polite. Make it more caveman."

Your coding agent adjusts the instruction, re-runs `agents-cli eval run`, and iterates until quality thresholds pass.

Deploy Agent

Tell your coding agent:

```
None
"Deploy this to Cloud Run"
```

Your coding agent activates the `google-agents-cli-deploy` skill and:

- Adds deployment infrastructure:

None

```
agents-cli scaffold enhance --deployment-target cloud_run
```

- Deploys:

None

```
agents-cli deploy
```

Your caveman agent is now live. Cloud Run URL in the output.

Observe Agent

Cloud Trace is enabled by default — no setup needed. Open the [Trace explorer](#) in the Google Cloud Console and send a few requests to your agent. You'll see spans for each LLM call and tool execution.

To go further and inspect the actual prompts and responses your agent handles in production, tell your coding agent:

None

```
"Set up observability infrastructure for my agent"
```

Your coding agent runs `infra single-project`, which provisions the service account, GCS bucket, and BigQuery dataset — and updates the deployed service to use them. See the [Observability Guide](#) for verification steps and advanced options.

Summary of Actions

Here's what each prompt triggered under the hood:

You said	Your coding agent did
"Build a caveman compressor agent"	Scaffolded project, wrote agent code, tested locally
"Write evals and run them"	Created evalset, configured LLM-as-judge, ran <code>agents-cli eval run</code>
"Deploy this to Cloud Run"	Added deployment target, deployed to Cloud Run
"Set up observability"	Provisioned service account, GCS bucket, and BigQuery dataset

The skills gave your coding agent the context to make the right decisions at each step — which ADK patterns to use, how to structure evals, which deploy target flags to pass.

Next steps

Try building something more complex:

- Add tools — "Add a Google Search tool so the caveman can grunt about current events"

- Multi-agent — "Create an A2A agent that other agents can talk to" (use [adk_a2a](#) template)
- RAG — "Build an agent that answers questions from our docs" (use [agentic_rag](#) template)

API Enablement & Configuration

To enable and configure the API access to Agent Platform, you must complete a sequence of steps including project setup, API activation, and identity management.

1. Initial Project Setup

Before enabling any APIs, you must have a valid Google Cloud environment. If you don't have a Google Cloud project, follow these steps to create a new project and enable billing.

- 1) **Create a Project:** Go to [Agent Platform on Google Cloud console](#). Click on the **project dropdown** at the top of the page and select **New Project**. Give your project a name and make a note of the **project ID**, which you will use later.

Note: Ensure you have the Project Creator role ([roles/resourcemanager.projectCreator](#)).

- 2) **Enable Billing:** Ensure you have an active billing account. This is necessary for using Agent Platform and other cloud services.
 - **Open the Billing Page:** Go to the [Google Cloud Billing page](#).
 - **Link a Billing Account:** If you do not have a billing account, you will be prompted to create one. If you have an existing billing account, you can link it to your project.
 - If you don't have an account, click **Manage billing account** to create a billing account:
 - Click **Create account**
 - Follow the prompts to enter your billing information and set up your account.
 - Associate billing account with your project:
 - Navigate back to the **Billing** section in the Google Cloud Console.
 - Select the project you created.
 - Choose or link your billing account to the project.

2. Install and initialize the Google Cloud CLI

The Google Cloud CLI will allow you to interact with Google Cloud services from the command line. You can use the **gcloud** command to enable APIs and authenticate your environment.

- Following the [Quickstart: Install the Google Cloud CLI](#) to install the Google Cloud **gcloud** CLI.
- Run **gcloud init** to initialize the gcloud CLI, enter your **project ID** when prompted. By default, this command opens a web browser to authorize access.

```
Shell
gcloud init
```

3. Enable APIs

You can enable the API through the Google Cloud console or the Google Cloud CLI **gcloud** command.

Option 1: Google Cloud console

- Go to [Agent Platform on Google Cloud console](#).
- Select your project from the Project selector,
- Click **Enable APIs** to enable required APIs.

Option 2: Google Cloud CLI

- Run this `gcloud` command to enable the `aiplatform.googleapis.com` service:

```
Shell
gcloud services enable aiplatform.googleapis.com
```

Note: Required Role: To enable APIs, you need the Service Usage Admin role (`roles/serviceusage.serviceUsageAdmin`).

4. Authenticate your environment

To access Google Cloud services, you must authenticate your requests. Choose the method that best fits your current development stage:

Method	Best For	How it Works
User Credentials and ADC (Recommended)	ADC is the secure, standard way to connect from company hardware or cloud environments.	You provide your user credentials to Application Default Credentials (ADC). ADC automatically authenticates requests to Google Cloud services.
Google Cloud API Keys	Quickest setup for local testing and rapid prototyping.	Google Cloud API keys work by acting as a project identifier rather than a user identifier.
Service Account	AI applications that are hosted on Google Cloud, on-premises, or on a different cloud provider.	You use a service account as the identity for your agent. To set up a VM and attach a service account for your AI application, you follow the specific workload setup instructions for your environment

Authenticate using user credentials and ADC

To use your own identity for **local development**, provide your user credentials to [Application Default Credentials](#) (ADC).

Option 1: gcloud

Running the Google Cloud CLI command `gcloud` creates a local credential file. ADC automatically finds and uses this file to authenticate requests to Google Cloud services, including Agent Platform servers.

Shell

```
gcloud auth application-default login
gcloud auth application-default set-quota-project [PROJECT_ID]
```

Replace [PROJECT_ID] with your project ID.

Option 2: Bash Script

Run the following bash script to set up ADC. Enter your project ID when prompted. This command opens a web browser to authorize access.

Shell

```
bash <(curl -sSL \
https://storage.googleapis.com/cloud-samples-data/adc/setup_adc.sh)
```

Option 3: Google Colab

If you access the API from a Google Colab, run the code below to authenticate your Colab environment.

Python

```
from google.colab import auth

auth.authenticate_user()
```

5. SDK Initialization and Configuration

Once the APIs are enabled and authentication is set, you can initialize the SDKs in your code.

Google Cloud provides distinct SDKs to interact with the Agent Platform ecosystem, each optimized for different workflows - ranging from traditional MLOps to generative AI and autonomous multi-agent orchestration.

SDK	Primary Focus	Package	Components	References
Google Gen AI SDK	Unified access to generative models including Gemini, Imagen, Veo.	google-genai	Gemini API and Gen AI models	Google Gen AI SDK
Vertex AI SDK	End-to-end machine learning lifecycle, MLOps, and custom infrastructure.	google-cloud-aiplat form	Model Training, Registry, Endpoints, Pipeline, etc.	Vertex AI SDK for Python Reference
ADK (Agent Development Kit)	Building, testing, and deploying autonomous multi-agent systems.	google-adk	Agents	ADK documentation

Google Gen AI SDK

- On your local machine, Install and update the Gen AI SDK for Python by running this command.

Shell

```
pip install --upgrade google-genai
```

- Initialize the Gen AI SDK client. Replace `PROJECT_ID` with your project ID.

Python

```
from google import genai
from google.genai import types

client = genai.Client(vertexai=True, project=PROJECT_ID, location="global")
```

Note: Alternatively you can set environment variables `GOOGLE_CLOUD_PROJECT`, `GOOGLE_CLOUD_LOCATION`, `GOOGLE_GENAI_USE_VERTEXAI`. The `client` will automatically pick up these environment variables in the `genai.Client()` initialization.

- Send a prompt to a Gemini model using Gemini API:

Python

```
response = client.models.generate_content(
    model="gemini-3.1-pro-preview",
    contents="How does AI work?",
)
print(response.text)
# Example response:
# Okay, let's break down how AI works. It's a broad field, so I'll focus on the ...
#
# Here's a simplified overview:
# ...
```

Vertex AI SDK

- On your local machine, Install and update the Vertex AI SDK for Python by running this command.

Shell

```
pip install --upgrade google-cloud-aiplatform
```

- Initialize the Gen AI SDK client. Replace `PROJECT_ID` with your project ID.

Python

```
from google.cloud import aiplatform

aiplatform.init(project=PROJECT_ID, location="us-central1")
```

- To deploy a model, you first need to create a [public endpoint](#). This example shows how to create a new public endpoint.

Python

```
endpoint = aiplatform.Endpoint.create(
    display_name="My Model Endpoint",
    project=PROJECT_ID,
    location="us-central1",
)
print(endpoint.resource_name)
```

- Once the endpoint is created, you can deploy a model to this endpoint using the SDK.

ADK

- On your local machine, Install and update the ADK by running this command.

Shell

```
pip install google-adk
```

- Create an agent project by running the `adk create` command to start a new agent project.

Shell

```
adk create my_agent
```

- **Explore the agent project.** The created agent project has the following structure, with the `agent.py` file containing the main control code for the agent.

None

```
my_agent/
  agent.py      # main agent code
  .env          # API keys or project IDs
  __init__.py
```

- **Update your agent project.** The `agent.py` file contains a `root_agent` definition which is the only required element of an ADK agent. You can also define tools for the agent to use. Update the generated `agent.py` code to include a `get_current_time` tool for use by the agent, as shown in the following code:

Python

```
from google.adk.agents.llm_agent import Agent

# Mock tool implementation
def get_current_time(city: str) -> dict:
    """Returns the current time in a specified city."""
    return {"status": "success", "city": city, "time": "10:30 AM"}

root_agent = Agent(
    model='gemini-3.1-flash-lite-preview',
    name='root_agent',
    description="Tells the current time in a specified city.",
    instruction="You are a helpful assistant that tells the current time in cities. Use the 'get_current_time' tool for this purpose.",
    tools=[get_current_time],
)
```

- Set the following environment variables in file `my_agent/.env`. Replace `PROJECT_ID` with your project ID.

None

```
GOOGLE_CLOUD_PROJECT=PROJECT_ID
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=TRUE
```

- **Run your agent.** You can run your ADK agent with an interactive command-line interface using the `adk run` command or the ADK web user interface provided by the ADK using the `adk web` command. Both these options allow you to test and interact with your agent.

Python

```
adk run my_agent
```

```
[user]: what can you do?
[root_agent]: I can tell you the current time in a specified city.
[user]: what time it is in London?
[root_agent]: The current time in London is 10:30 AM.
[user]:
```

- Run with the web interface. The ADK framework provides a web interface you can use to test and interact with your agent. You can start the web interface using the following command:

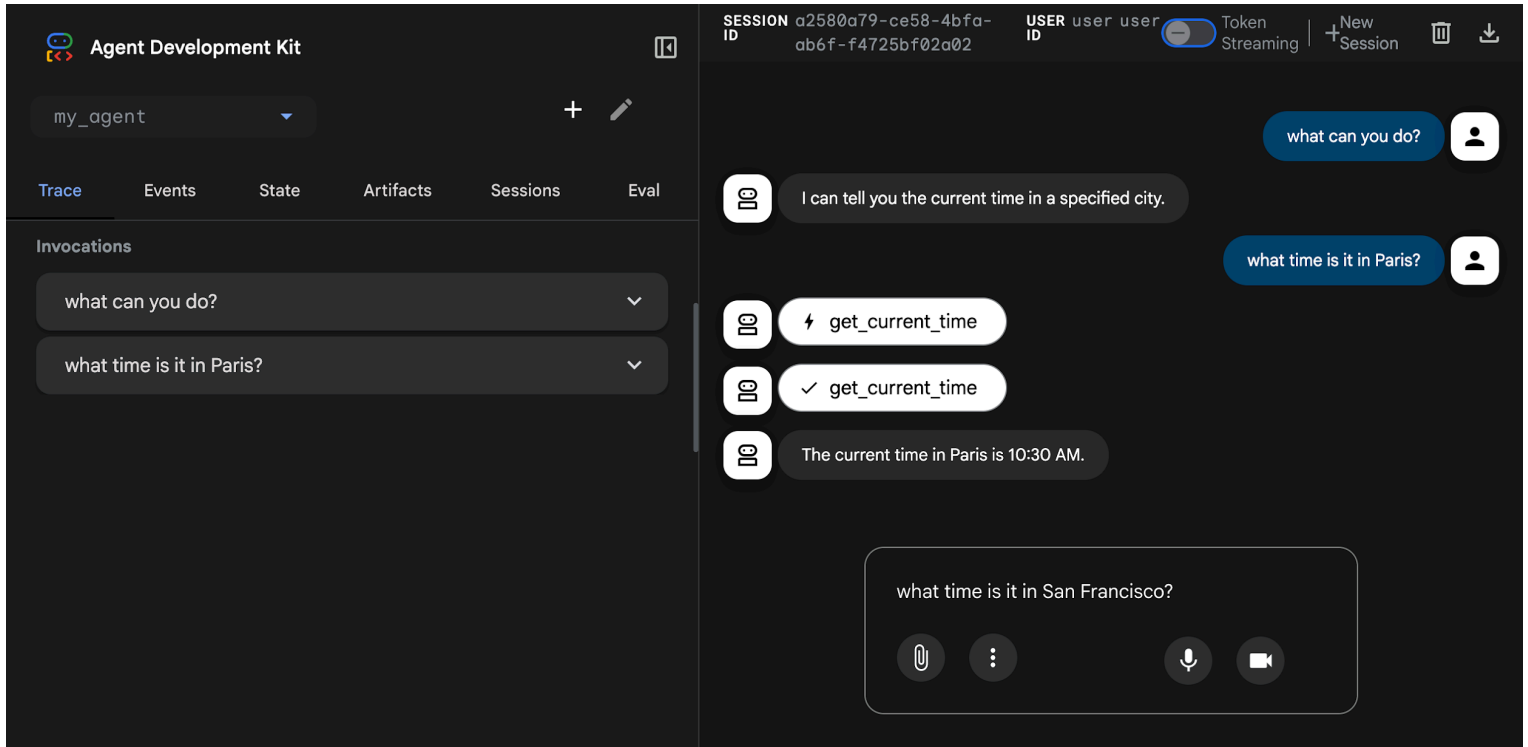
Python

```
adk web --port 8000
```

Note:

Run this command from the **parent directory** that contains your **my_agent/** folder. For example, if your agent is inside **agents/my_agent/**, run **adk web** from the **agents/** directory.

This command starts a web server with a chat interface for your agent. You can access the web interface at (<http://localhost:8000>). Select the agent at the upper left corner and type a request.



- Now that you have ADK installed and your first agent running.

Additional Resources

- [Blog](#)
- [Documentation](#)
- [Console](#)
- [Intro video](#)
- [Repo](#)